Ready.

COURSE CODE: CP-1259

Python Programming

📚 Diploma in Computer Engineering | Semester 2

🕐 Total Hours: 75 | Credits: 4 | Weeks: 15

📅 Revision Date: 27 July 2026

Interactive Lesson with Malayalam Notes • Calculators • Visual Guides • Practice Problems

📋 Source Declaration

This lesson file consolidates the **Python Programming** syllabus (CP-1259) for Diploma in Computer Engineering. The syllabus data is sourced from the official curriculum document. All four modules are covered with detailed explanations, code examples, and bilingual Malayalam-English notes.

Sparse Experiment Details: Some advanced subtopics (e.g., detailed regex flags, all built-in exceptions, advanced metaclasses) are treated as "sparse" — core concepts are fully explained while exhaustive enumeration is deferred to the *References* section. The focus is on *teachable depth* suitable for diploma-level students.

> 📌 **കുറിപ്പ്:** ഈ പാഠഭാഗം ഔദ്യോഗിക സിലബസ് അനുസരിച്ച് തയ്യാറാക്കിയതാണ്. എല്ലാ നാല് മൊഡ്യൂളുകളും പൂർണ്ണമായും ഉൾപ്പെടുത്തിയിട്ടുണ്ട്. സാങ്കേതിക പദങ്ങൾ ഇംഗ്ലീഷിൽ തന്നെ സൂക്ഷിച്ചിരിക്കുന്നു.

📊 Course Dashboard

Module	Topic	Hours	Weightage	Status
1	Python Basics — Data Types, Operators, Control Flow, Strings	18	25%	✅ Complete
2	Data Structures — Lists, Tuples, Dictionaries, Sets	18	25%	✅ Complete
3	Functions & Modules — Functions, Recursion, File I/O, Modules	20	25%	✅ Complete
4	Advanced Topics — OOP, Exception Handling, Regex, Debugging	19	25%	✅ Complete
Total			**100%**	✅

📖 How to Use This Lesson

1. **Start with the Roadmap:** Understand the learning path.
2. **Read each Module sequentially:** Modules build upon each other.
3. **Try the code examples:** Copy-paste into your Python interpreter or IDE.
4. **Use the calculators:** Interactive tools to test concepts.
5. **Check Malayalam notes (.ml-note boxes):** For conceptual clarity in your native language.
6. **Attempt Practice Questions:** At the end of each module and in the dedicated section.
7. **Use Search (🔍):** To quickly find any topic, function, or concept.
8. **Print or Download:** For offline study.

> 📌 **ഉപയോഗിക്കുന്ന വിധം:** ഓരോ മൊഡ്യൂളും ക്രമമായി പഠിക്കുക. കോഡ് ഉദാഹരണങ്ങൾ സ്വയം ടൈപ്പ് ചെയ്ത് റൺ ചെയ്യുക. കാൽക്കുലേറ്ററുകൾ ഉപയോഗിച്ച് ആശയങ്ങൾ പരീക്ഷിക്കുക.

🎯 Course Outcomes (COs)

Upon completion of this course, the student will be able to:

> **CO1:** Write, debug, and execute Python programs using basic syntax, data types, and operators.

> **CO2:** Implement control flow using conditionals and loops, and manipulate strings effectively.

> **CO3:** Utilize Python's built-in data structures — lists, tuples, dictionaries, and sets — to solve problems.

> **CO4:** Design modular programs using functions, recursion, modules, and handle file I/O operations.

> **CO5:** Apply object-oriented programming principles including classes, inheritance, and polymorphism.

> **CO6:** Handle errors using exception handling, use regular expressions, and apply debugging techniques.

🗺️ Syllabus Coverage Map

Module	Topics Covered	Depth
1	Introduction, Identifiers, Data Types, Operators, `if-elif-else`, `for` / `while` loops, `break` / `continue` / `pass`, String methods, String formatting	Deep
2	Lists (creation, methods, slicing, comprehensions), Tuples (immutability, packing/unpacking), Dictionaries (keys, values, methods, dict comprehensions), Sets (operations, methods)	Deep

Module	Topics Covered	Depth
3	Function definition, arguments (default, keyword, *args, **kwargs), scope, recursion, lambda, map/filter/reduce, modules (`import` , `math` , `random` , `datetime`), File I/O (read, write, append, `with`)	Deep
4	OOP (classes, objects, `__init__` , inheritance, polymorphism, encapsulation), Exception handling (`try-except-finally`), Regular expressions (`re` module), Debugging (`pdb` , logging, assertions)	Deep (regex & pdb sparse)

🧭 Learning Roadmap

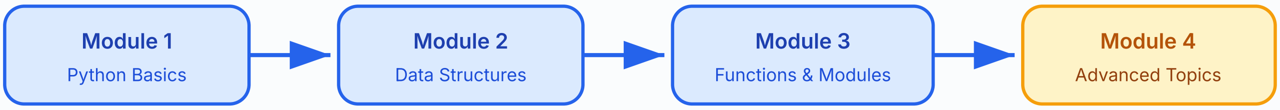

Figure: Sequential learning path through the 4 modules of Python Programming.

🔰 Module 1: Python Basics

Hours: 18 | **Topics:** Introduction, Data Types, Operators, Control Flow, Strings

1.1 Introduction to Python

Python is a high-level, interpreted, dynamically-typed programming language created by **Guido van Rossum** in 1991. It emphasises code readability with its use of significant whitespace (indentation).

> 📌 **പൈത്തൺ പരിചയം:** വളരെ ലളിതമായി പഠിക്കാവുന്ന ഒരു പ്രോഗ്രാമിംഗ് ഭാഷയാണ് പൈത്തൺ. ഇൻഡന്റേഷൻ വളരെ പ്രധാനമാണ് — ബ്രാക്കറ്റുകൾക്ക് പകരം സ്പേസുകൾ ഉപയോഗിച്ചാണ് കോഡ് ബ്ലോക്കുകൾ നിർവചിക്കുന്നത്.

```
# Your first Python program
print("Hello, World!")  # Output: Hello, World!

# Python is dynamically typed
x = 42          # integer
x = "hello"     # now a string — perfectly valid!
```

1.2 Identifiers & Data Types

Identifiers are names given to variables, functions, classes, etc. Rules: must start with a letter or underscore, can contain letters/digits/underscores, case-sensitive, and cannot be a reserved keyword.

Core Data Types:

Type	Example	Mutable?	Description
`int`	`42`	❌	Integer (unlimited precision)
`float`	`3.14`	❌	Floating-point number
`str`	`"hello"`	❌	String (sequence of characters)
`bool`	`True`	❌	Boolean (`True` / `False`)
`list`	`[1,2,3]`	✅	Ordered, mutable collection
`tuple`	`(1,2,3)`	❌	Ordered, immutable collection
`dict`	`{"a":1}`	✅	Key-value pairs
`set`	`{1,2,3}`	✅	Unordered, unique elements

1.3 Operators

Python supports arithmetic (`+ - * / // % **`), comparison (`== ≠ < > ⩽ ⩾`), logical (`and or not`), assignment (`= += -= *= /=`), identity (`is is not`), and membership (`in not in`) operators.

```
a, b = 10, 3
print(a + b)    # 13
print(a ** b)   # 1000 (10³)
print(a / b)    # 3.333...
print(a // b)   # 3 (floor division)
print(a % b)    # 1 (modulus)

# Logical operators
print(a > 5 and b < 5)  # True
print(not False)         # True

# Membership
fruits = ["apple", "banana", "mango"]
print("banana" in fruits)      # True
print("grape" not in fruits)   # True
```

1.4 Control Flow — Conditionals

```
score = 85
if score ⩾ 90:
    grade = "A"
elif score ⩾ 80:
    grade = "B"
elif score ⩾ 70:
    grade = "C"
else:
    grade = "F"
print(f"Grade: {grade}")  # Grade: B
```

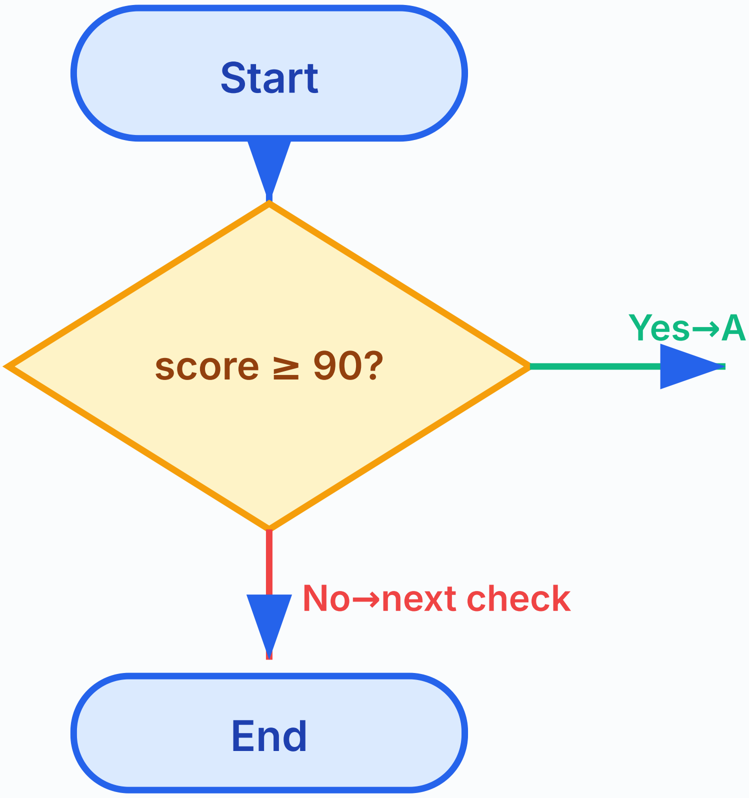

Figure: Flowchart of if-elif-else conditional logic for grade assignment.

1.5 Loops — `for` and `while`

```
# for loop with range
for i in range(1, 6):
    print(i, end=" ")  # Output: 1 2 3 4 5

# Iterating over a list
colors = ["red", "green", "blue"]
for color in colors:
    print(color.upper())  # RED GREEN BLUE

# while loop
count = 0
while count < 5:
    print(count, end=" ")
    count += 1
# Output: 0 1 2 3 4

# break and continue
for num in range(10):
    if num == 3:
        continue  # skip 3
    if num == 7:
        break     # stop at 7
    print(num, end=" ")
# Output: 0 1 2 4 5 6
```

⚠️ **Common Mistake:** Forgetting to update the loop variable in a `while` loop leads to an **infinite loop**. Always ensure the condition eventually becomes `False` .

1.6 Strings

Strings are immutable sequences of Unicode characters. Python provides rich string methods.

```
text = "  Hello, Python World!  "
print(text.strip())           # "Hello, Python World!"
print(text.lower())           # "  hello, python world!  "
print(text.upper())           # "  HELLO, PYTHON WORLD!  "
print(text.replace("Python", "Java"))  # "  Hello, Java World!  "
print(text.split(","))        # ['  Hello', ' Python World!  ']
print("-".join(["a","b","c"]))  # "a-b-c"

# String formatting
name, age = "Aarav", 20
print(f"My name is {name} and I am {age} years old.")
# Output: My name is Aarav and I am 20 years old.

# Slicing
s = "Python"
print(s[0:3])   # "Pyt"
print(s[::-1])  # "nohtyP" (reversed)
```

📦 Module 2: Data Structures

Hours: 18 | **Topics:** Lists, Tuples, Dictionaries, Sets

2.1 Lists

Lists are **ordered, mutable** collections. They are the workhorse data structure in Python.

```
# Creating lists
empty_list = []
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]

# List methods
numbers.append(6)         # [1,2,3,4,5,6]
numbers.insert(0, 0)      # [0,1,2,3,4,5,6]
numbers.remove(3)          # [0,1,2,4,5,6]
popped = numbers.pop()     # popped=6, list=[0,1,2,4,5]
numbers.sort(reverse=True) # [5,4,2,1,0]
print(len(numbers))        # 5

# Slicing (creates a new list)
print(numbers[1:4])        # [4,2,1]

# List comprehension
squares = [x**2 for x in range(1, 6)]
print(squares)             # [1, 4, 9, 16, 25]

# Nested lists (2D)
matrix = [[1,2,3], [4,5,6], [7,8,9]]
print(matrix[1][2])        # 6
```

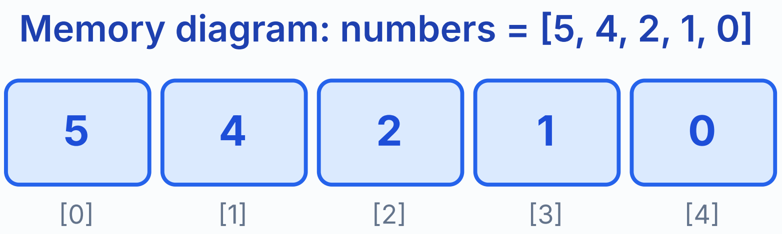

Figure: Memory layout of a list — each element stored at a contiguous index.

2.2 Tuples

Tuples are **ordered, immutable** sequences. Once created, they cannot be modified.

```
# Creating tuples
point = (3, 4)
single_item = (42,)     # comma is essential!
empty = ()

# Tuple unpacking
x, y = point
print(x, y)             # 3 4

# Why tuples? Faster than lists, hashable (can be dict keys), safe from modification
coordinates = {(0,0): "origin", (1,0): "right"}
print(coordinates[(1,0)])  # "right"

# Converting between list and tuple
my_list = [1, 2, 3]
my_tuple = tuple(my_list)
back_to_list = list(my_tuple)
```

⚠️ **Common Mistake:** `t = (42)` is NOT a tuple — it's just the integer `42` in parentheses. Use `t = (42,)` with a trailing comma for a single-element tuple.

2.3 Dictionaries

Dictionaries store **key-value pairs**. Keys must be immutable (strings, numbers, tuples). Values can be anything.

```
# Creating dictionaries
student = {"name": "Aarav", "age": 20, "grade": "A"}
empty_dict = {}
using_constructor = dict(name="Priya", age=21)

# Access and modify
print(student["name"])        # "Aarav"
print(student.get("city", "N/A"))  # "N/A" (safe access)
student["age"] = 21
student["city"] = "Kochi"

# Dictionary methods
print(student.keys())        # dict_keys(['name','age','grade','city'])
print(student.values())      # dict_values(['Aarav',21,'A','Kochi'])
print(student.items())       # dict_items([('name','Aarav'),...])
```

```
# Dictionary comprehension
squares_dict = {x: x**2 for x in range(1, 6)}
print(squares_dict)          # {1:1, 2:4, 3:9, 4:16, 5:25}

# Iterating
for key, value in student.items():
    print(f"{key} → {value}")
```

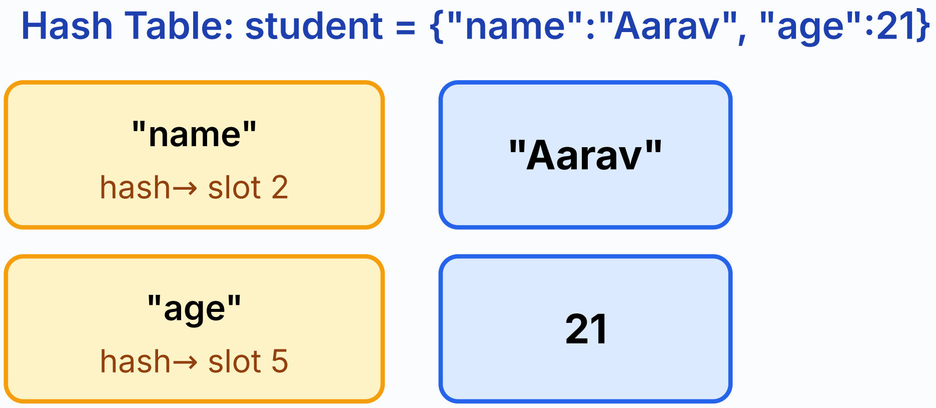

Figure: Dictionary uses a hash table — keys are hashed to determine storage slots for O(1) average lookup.

2.4 Sets

Sets are **unordered, mutable** collections of **unique, hashable** elements. Great for membership testing and eliminating duplicates.

```
# Creating sets
fruits = {"apple", "banana", "mango"}
empty_set = set()  # NOT {} — that creates a dict!
nums = set([1, 2, 2, 3, 3, 4])
print(nums)  # {1, 2, 3, 4} — duplicates removed!

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)   # Union: {1,2,3,4,5,6}
print(a & b)   # Intersection: {3,4}
print(a - b)   # Difference: {1,2}
print(a ^ b)   # Symmetric difference: {1,2,5,6}

# Set methods
fruits.add("grape")
fruits.remove("banana")
print("mango" in fruits)  # True (fast O(1) lookup)
```

⚙️ Module 3: Functions & Modules

Hours: 20 | **Topics:** Functions, Recursion, Lambda, Modules, File I/O

3.1 Defining Functions

```python
def greet(name, greeting="Hello"):
    """Return a greeting string."""  # docstring
    return f"{greeting}, {name}!"

print(greet("Aarav"))              # Hello, Aarav!
print(greet("Priya", "Namaste"))  # Namaste, Priya!

# *args and **kwargs
def sum_all(*args):
    """Accept any number of positional arguments."""
    return sum(args)

print(sum_all(1, 2, 3, 4, 5))     # 15

def print_info(**kwargs):
    """Accept any number of keyword arguments."""
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Aarav", age=20, city="Kochi")
```

3.2 Scope & Lifetime

Python follows the **LEGB** rule: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

```python
x = "global"  # global scope

def outer():
    x = "enclosing"  # enclosing scope
    def inner():
        x = "local"  # local scope
        print(f"inner: {x}")
    inner()
    print(f"outer: {x}")

outer()
print(f"global: {x}")
# Output:
# inner: local
# outer: enclosing
# global: global
```

⚠️ **Common Mistake:** Using a mutable default argument like `def f(lst=[]):` — the same list object is shared across all calls! Use `def f(lst=None): if lst is None: lst = []` instead.

3.3 Recursion

```python
def factorial(n):
    """Return n! using recursion."""
    if n <= 1:
        return 1  # base case
    return n * factorial(n - 1)  # recursive case

print(factorial(5))  # 120

# Tracing factorial(3):
# factorial(3) → 3 * factorial(2)
#   factorial(2) → 2 * factorial(1)
```

```
#     factorial(1) → 1 (base case)
# Unwinding: 1 → 2*1=2 → 3*2=6
```

```
📤 Output Trace:
                factorial(3) = 3 * factorial(2)
                = 3 * (2 * factorial(1))
                = 3 * (2 * 1)
                = 3 * 2 = 6
```

3.4 Lambda & Higher-Order Functions

```python
# Lambda: anonymous single-expression function
square = lambda x: x ** 2
print(square(5))  # 25

# map: apply function to every element
nums = [1, 2, 3, 4, 5]
doubled = list(map(lambda x: x * 2, nums))
print(doubled)  # [2, 4, 6, 8, 10]

# filter: keep elements that satisfy condition
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)  # [2, 4]

# reduce: cumulative operation (needs functools)
from functools import reduce
total = reduce(lambda acc, x: acc + x, nums, 0)
print(total)  # 15
```

3.5 Modules

```python
# Importing standard library modules
import math
print(math.sqrt(16))    # 4.0
print(math.pi)          # 3.141592653589793

from random import randint, choice
print(randint(1, 100))  # random integer 1-100
print(choice(["red", "green", "blue"]))  # random pick

import datetime
now = datetime.datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))
# Output: 2026-07-27 14:30:00 (example)
```

3.6 File I/O

```python
# Writing to a file
with open("sample.txt", "w", encoding="utf-8") as f:
    f.write("Hello, Python!\n")
    f.write("Line 2\n")

# Reading from a file
with open("sample.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)

# Reading line by line
```

```
with open("sample.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())

# Appending
with open("sample.txt", "a", encoding="utf-8") as f:
    f.write("Appended line\n")

# The 'with' statement auto-closes the file — no need for f.close()
```

📌 **ഫയൽ I/O:** `with` സ്റ്റേറ്റ്മെന്റ് ഉപയോഗിക്കുന്നതാണ് ഏറ്റവും സുരക്ഷിതം. ഇത് ഫയൽ ഓട്ടോമാറ്റിക്കായി ക്ലോസ് ചെയ്യും — `f.close()` വിളിക്കാൻ മറന്നാലും പ്രശ്നമില്ല.

🚀 Module 4: Advanced Topics

Hours: 19 | **Topics:** OOP, Exception Handling, Regex, Debugging

4.1 Object-Oriented Programming (OOP)

```
class Animal:
    """Base class for all animals."""
    species_count = 0  # class variable

    def __init__(self, name, species):
        self.name = name        # instance variable
        self.species = species
        Animal.species_count += 1

    def speak(self):
        return f"{self.name} makes a sound."

    @classmethod
    def get_count(cls):
        return cls.species_count

    @staticmethod
    def kingdom():
        return "Animalia"

# Inheritance
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, "dog")  # call parent __init__
        self.breed = breed

    def speak(self):  # override
        return f"{self.name} barks!"

# Polymorphism
animals = [Animal("Generic", "unknown"), Dog("Bruno", "Labrador")]
for a in animals:
    print(a.speak())
# Output:
# Generic makes a sound.
# Bruno barks!

print(Animal.get_count())  # 2
print(Animal.kingdom())    # Animalia
```

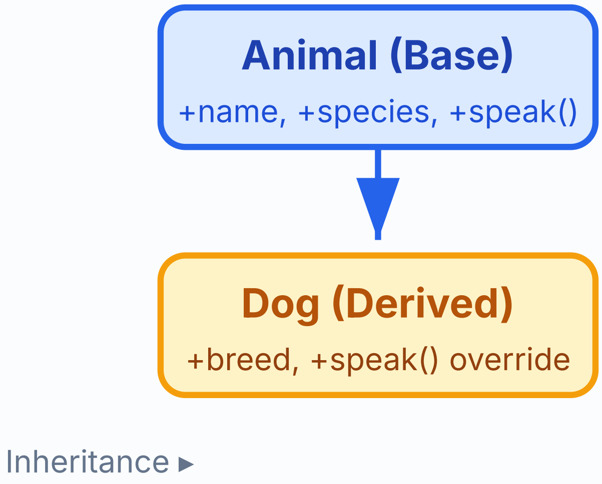

Figure: Inheritance hierarchy — Dog inherits from Animal and overrides speak().

4.2 Exception Handling

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None
    except TypeError:
        print("Error: Invalid types for division.")
        return None
    else:
        print("Division successful!")
        return result
    finally:
        print("Cleanup: safe_divide finished.")

print(safe_divide(10, 2))   # 5.0
print(safe_divide(10, 0))   # None
# Output:
# Division successful!
# Cleanup: safe_divide finished.
# 5.0
# Error: Cannot divide by zero!
# Cleanup: safe_divide finished.
# None
```

4.3 Regular Expressions (Sparse)

```
import re

text = "Contact: aarav@email.com or priya123@college.edu"

# Find all email patterns
pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
emails = re.findall(pattern, text)
print(emails)  # ['aarav@email.com', 'priya123@college.edu']
```

```
# Search and replace
cleaned = re.sub(r'\d+', 'XXX', "Phone: 9876543210")
print(cleaned)  # Phone: XXX

# Match validation
phone_pattern = r'^\d{10}$'
print(re.match(phone_pattern, "9876543210"))  # Match object
print(re.match(phone_pattern, "12345"))       # None
```

Note: Advanced regex flags and lookahead/lookbehind assertions are considered sparse topics — refer to Python `re` module documentation for exhaustive coverage.

4.4 Debugging Techniques

```
# Using assertions
def calculate_average(numbers):
    assert len(numbers) > 0, "List cannot be empty"
    return sum(numbers) / len(numbers)

# Using logging
import logging
logging.basicConfig(level=logging.DEBUG,
                    format='%(levelname)s:%(message)s')

def process_data(data):
    logging.debug(f"Processing {data}")
    result = [x * 2 for x in data]
    logging.info(f"Result: {result}")
    return result

process_data([1, 2, 3])
# Output:
# DEBUG:Processing [1, 2, 3]
# INFO:Result: [2, 4, 6]

# pdb (Python Debugger) — sparse intro
# import pdb; pdb.set_trace()  # set breakpoint
```

> 📌 **ഡീബഗ്ഗിംഗ്:** `print()` ഉപയോഗിച്ച് ഡീബഗ് ചെയ്യുന്നതിനു പകരം `logging` മൊഡ്യൂൾ ഉപയോഗിക്കുന്നതാണ് നല്ല ശീലം. ഇത് ലെവൽ അനുസരിച്ച് മെസേജുകൾ ഓൺ/ഓഫ് ചെയ്യാൻ സഹായിക്കുന്നു.

📐 Formula & Syntax Bank

Concept	Syntax / Formula	Example
List comprehension	`[expr for item in iterable if condition]`	`[x**2 for x in range(5) if x%2==0]`
Dict comprehension	`{k: v for k,v in iterable}`	`{x:x**2 for x in range(3)}`
Lambda	`lambda args: expression`	`lambda a,b: a+b`
Ternary operator	`val_if_true if cond else val_if_false`	`"Even" if n%2==0 else "Odd"`
f-string	`f"text {variable}"`	`f"Age: {age}"`

Concept	Syntax / Formula	Example
File open	`with open(path, mode) as f:`	`with open("f.txt","r") as f:`
Exception	`try: ... except X as e: ... finally: ...`	See Module 4

🖼️ Visual Library

All diagrams in this lesson are embedded as inline SVGs for clarity and accessibility.

- **Figure 1:** Learning Roadmap — sequential module flow (see Roadmap)
- **Figure 2:** if-elif-else Flowchart (see Module 1.4)
- **Figure 3:** List Memory Layout (see Module 2.1)
- **Figure 4:** Dictionary Hash Table (see Module 2.3)
- **Figure 5:** OOP Inheritance Hierarchy (see Module 4.1)

🧪 Interactive Activities & Calculators

🎓 Grade Calculator

Enter a score (0–100) to see the grade using if-elif-else logic.

Score: e.g. 85

Result will appear here...

🔢 Factorial Calculator (Recursive)

Enter a non-negative integer to compute its factorial.

n: e.g. 5

Result will appear here...

⚖️ BMI Calculator

Enter weight (kg) and height (m) to calculate BMI.

Weight (kg): e.g. 70 Height (m): e.g. 1.75

Result will appear here...

📝 Assessment Scheme

Component	Weightage	Type
Internal Assessment (IA)	40%	Assignments, quizzes, lab work, attendance
End Semester Exam (ESE)	60%	Written theory + programming problems
Total	**100%**	Minimum pass: 40% overall

❓ Practice Questions

Module 1

1. Write a program to check if a given number is prime.
2. Explain the difference between `break`, `continue`, and `pass` with examples.
3. Write a program to reverse a string without using `[::-1]`.

Module 2

1. Create a list of 10 numbers and write code to find the second largest element.
2. Given two dictionaries, merge them into one. If keys overlap, sum their values.
3. Demonstrate set operations (union, intersection, difference) with real-world examples.

Module 3

1. Write a recursive function to compute the nth Fibonacci number.
2. Explain `*args` and `**kwargs` with a practical example.
3. Write a program to count word frequency in a text file.

Module 4

1. Design a `BankAccount` class with deposit, withdraw, and balance-check methods.
2. Write a function that uses exception handling to safely parse an integer from user input.
3. Use regex to validate Indian mobile numbers and email addresses.

📄 Model Question Paper

Duration: 3 Hours | **Max Marks:** 100

Part	Questions	Marks
A	10 short-answer (all compulsory)	10 × 2 = 20
B	5 out of 7 medium-length	5 × 8 = 40
C	2 out of 4 long programming problems	2 × 20 = 40

Sample Questions:

- **Part A:** Define tuple unpacking. What is the output of `print(2**3**2)`?
- **Part B:** Explain dictionary methods with examples. Write a program using list comprehension to filter even numbers.
- **Part C:** Design a complete class hierarchy for a Library Management System. Write a Python program to read a CSV file and generate a summary report.

🔄 Revision Notes

Module 1 Quick Recap

- Python is dynamically typed and uses indentation for blocks.
- Core types: int, float, str, bool, list, tuple, dict, set.
- `if-elif-else` for branching; `for` and `while` for loops.
- Strings are immutable; use methods like `.strip()`, `.split()`, `.join()`.

Module 2 Quick Recap

- Lists: mutable, ordered. Use `.append()`, slicing, comprehensions.
- Tuples: immutable, ordered. Good for fixed data and dict keys.
- Dicts: key-value pairs. Use `.get()` for safe access.
- Sets: unique elements. Use `|`, `&`, `-`, `^` for set operations.

Module 3 Quick Recap

- Functions: `def`, default args, `*args`, `**kwargs`.
- Scope: LEGB rule (Local→Enclosing→Global→Built-in).
- Recursion needs a base case to terminate.
- File I/O: use `with open(...) as f:` for auto-closing.

Module 4 Quick Recap

- OOP: Classes, `__init__`, inheritance, polymorphism, encapsulation.
- Exceptions: `try-except-else-finally`.

- Regex: `re.findall()`, `re.sub()`, `re.match()`.
- Debugging: Use `logging` instead of `print()`.

📚 Glossary

Argument: A value passed to a function when it is called.

Class: A blueprint for creating objects, defining attributes and methods.

Dictionary: An unordered, mutable collection of key-value pairs with O(1) average lookup.

Exception: An error that occurs during program execution, which can be caught and handled.

Immutable: An object whose state cannot be modified after creation (e.g., str, tuple, int).

Lambda: An anonymous, single-expression function defined with the `lambda` keyword.

List Comprehension: A concise way to create lists using a single line of code: `[expr for item in iterable]`.

Module: A file containing Python definitions and statements that can be imported.

Mutable: An object whose state can be modified after creation (e.g., list, dict, set).

Recursion: A technique where a function calls itself to solve smaller instances of the same problem.

🔗 References

- Python Official Documentation: docs.python.org/3/
- Python Standard Library: docs.python.org/3/library/
- PEP 8 — Style Guide: peps.python.org/pep-0008/
- Real Python Tutorials: realpython.com
- W3Schools Python: w3schools.com/python/
- GeeksforGeeks Python: geeksforgeeks.org